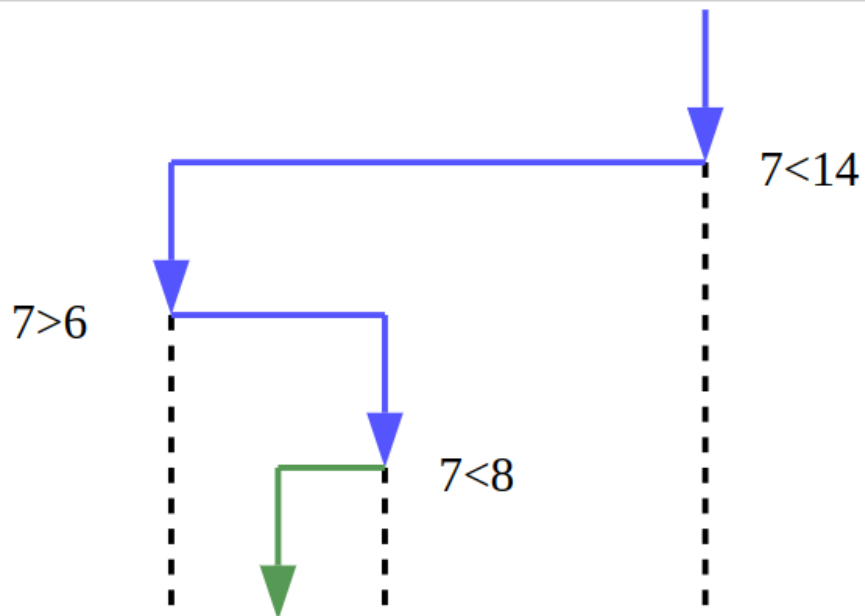


Python

11

Линейный VS бинарный поиск



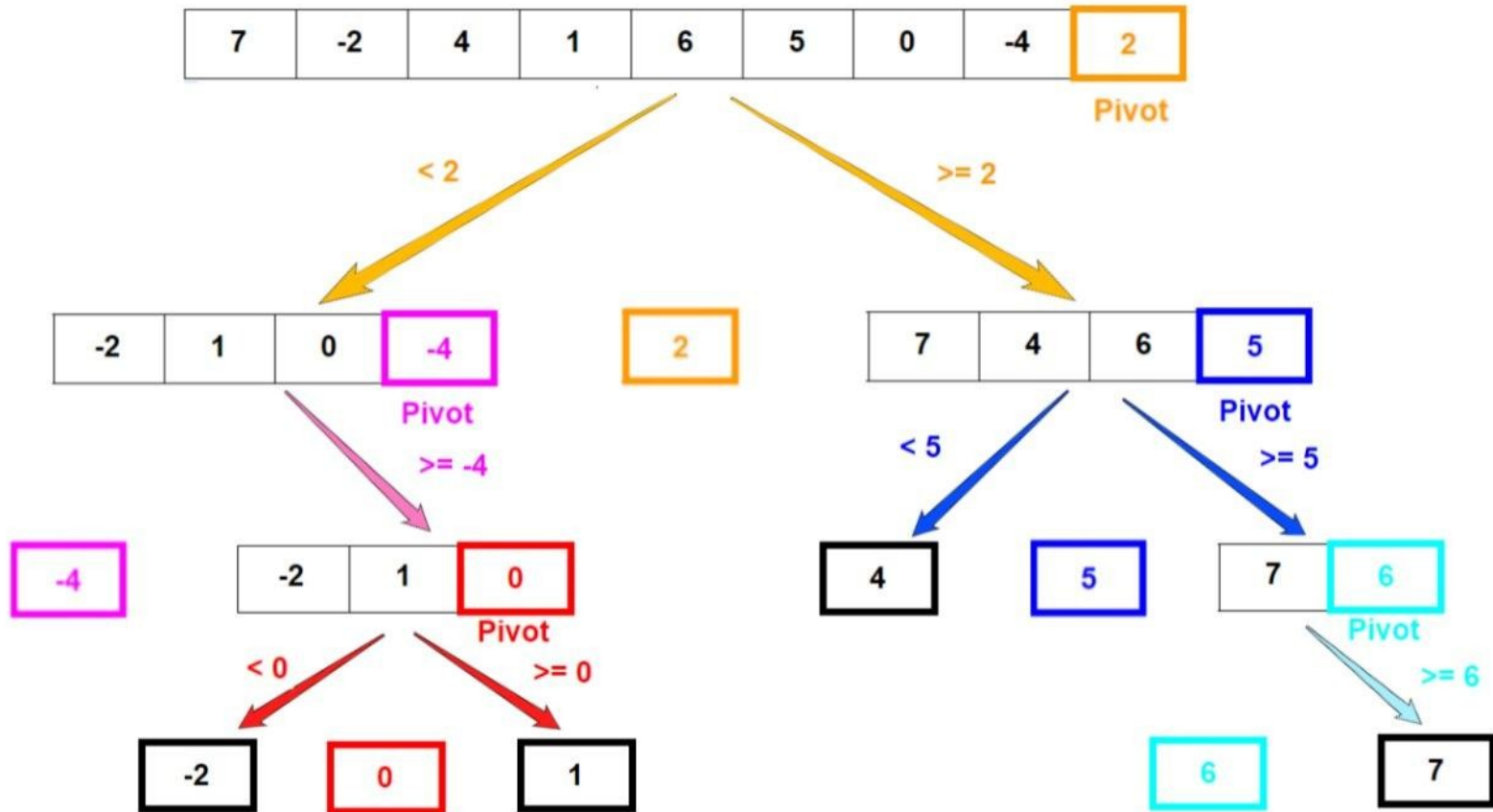
0	1	3	4	6	7	8	10	13	14	18	19	21	24	37	40	45	71
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	

bubble сортировка

исходный массив		обмен 2 и 3		обмен 2 и 7		обмен 2 и 5		нет обмена
1		1		1		1		1
5		5		5		2		2
7		7		2		5		5
3		2		7		7		7
2		3		3		3		3

первый проход циклом по массиву

quick сортировка



Режим	Обозначение
'r'	открытие на чтение (является значением по умолчанию).
'w'	открытие на запись, содержимое файла удаляется, если файла не существует, создается новый.
'x'	открытие на запись, если файла не существует, иначе исключение.
'a'	открытие на дозапись, информация добавляется в конец файла.
'b'	открытие в двоичном режиме.
't'	открытие в текстовом режиме (является значением по умолчанию).
'+'	открытие на чтение и запись

Закриття файлу

Спосіб №1

```
f = open('example.txt','r')  
# робота з файлом  
f.close()
```

Спосіб №2

```
f = open('example.txt','r')  
try:  
    # робота з файлом  
finally:  
    f.close()
```

Спосіб №3

```
with open('example.txt') as f:  
    # робота з файлом
```

Читання та запис файлів у Python

Функція read()

```
f = open('example.txt','r')  
f.read(7) # читання 7 символів з  
          # example.txt  
f.read()  # читання всього файлу
```

Функція readline()

```
f = open('test.txt','r')  
f.readline() # прочитати перший рядок  
f.readlines() # прочитати всі рядки
```

Функція write()

```
f = open('xyz.txt','w') # відкриття в режимі запису  
f.write('Hello \n World') # запис Hello World у файл  
f.close()                # закриття файлу
```

Що таке клас

Клас є шаблоном або формальним описом об'єкта, а об'єкт представляє екземпляр цього класу, його реальне втілення.

Основні парадигми ООП

Абстракція - Інкапсуляція - Спадкування - Поліморфізм

Інкапсуляція

Інкапсуляція - обмеження доступу до складових об'єкта (методів, змінних і т.д.)

Інкапсуляція в **Python** працює лише на рівні угоди. Це означає, що викликати якісь методи із класу не можна, але **Python** дозволяє це робити інакше.

```
1 class A:
2     def _private(self):
3         print("Это приватный метод!")
```

```
1 >>a = A()
2 >>a._private()
3 Это приватный метод!
```

```
1 class B:
2     def __private(self):
3         print("Это приватный метод!")
```

```
1 Traceback (most recent call last):
2   File "<input>", line 1, in <module>
3   AttributeError: 'B' object has no attribute '__private'
```

```
1 >>b._B__private()
2 Это приватный метод!
```

Спадкування

Спадкування передбачає те, що дочірній клас містить усі атрибути батьківського класу, причому деякі з них можуть бути перевизначені або додані (розширені) в дочірньому класі.

```
1 class Human:
2     def breathe(self):
3         print("Человек дышит")
4
5     def walk(self):
6         print("Человек ходит")
7
8
9 class Doctor(Human):
10     def heal(self):
11         print("Доктор лечит")
```

```
1 >>d = Doctor()
2 >>d.walk()
3 Человек ходит
4 >>d.breathe()
5 Человек дышит
6 >>d.heal()
7 Доктор лечит
```

```
1 class Doctor(Human):
2     def heal(self):
3         print("Доктор лечит")
4     def walk(self):
5         print("Доктор ходит")
```

```
1 >>d.walk()
2 Доктор ходит
3
4 h = Human()
5 >>h.walk()
6 Человек ходит
```

Поліморфізм

Поліморфізм полягає у використанні єдиної сутності (метод, оператор або об'єкт) для подання різної поведінки в різних сценаріях використання

```
1 class Cat:
2     def info(self):
3         print(f"I am a cat.")
4
5     def make_sound(self):
6         print("Meow")
7
8
9 class Dog:
10    def info(self):
11        print(f"I am a dog")
12
13    def make_sound(self):
14        print("Wow")
```

```
1 >>c = Cat()
2 >>d = Dog()
3 for animal in (c, d):
4     animal.make_sound()
5     animal.info()
6     animal.make_sound()
7 Meow
8 I am a cat.
9 Meow
10 Wow
11 I am a dog
12 Wow
```

Абстракція

Абстракція - використання лише тих характеристик об'єкта, які з достатньою точністю представляють його у цій системі. Основна ідея полягає в тому, щоб представити об'єкт мінімальним набором полів та методів і при цьому з достатньою точністю для задачі, що розв'язується.

Абстракція дозволяє працювати з об'єктами не вдаючись до подробиць їх реалізації.